

Introduction to Algorithm

기초 알고리즘

Backtracking 개요



백트래킹 (Backtracking)

완전탐색 알고리즘의 시간 개선을 위한 알고리즘을
한 가지 알아보도록 하자



백트래킹 (Backtracking)

만약, 순열을 구해야 하는 문제가 주어진다면?

재귀함수를 활용할 수 있음



백트래킹 (Backtracking)

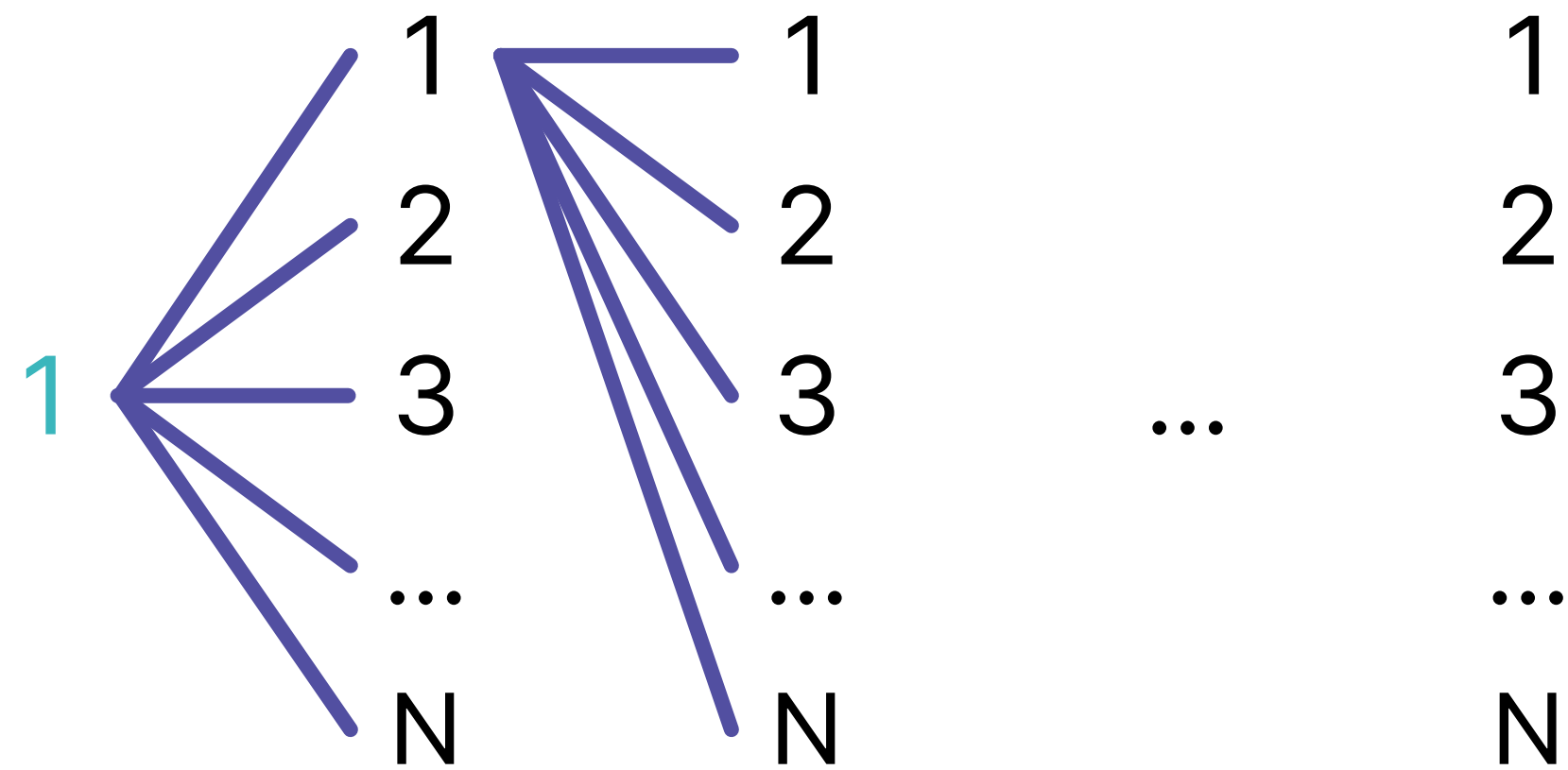
만약 10개의 원소에서 9개를 재귀를 통해 고른다면?

$$10^9 = 1,000,000,000$$

다만 완전탐색 알고리즘의 한계로,
순열의 길이가 길어지면 많은 시간이 소요됨



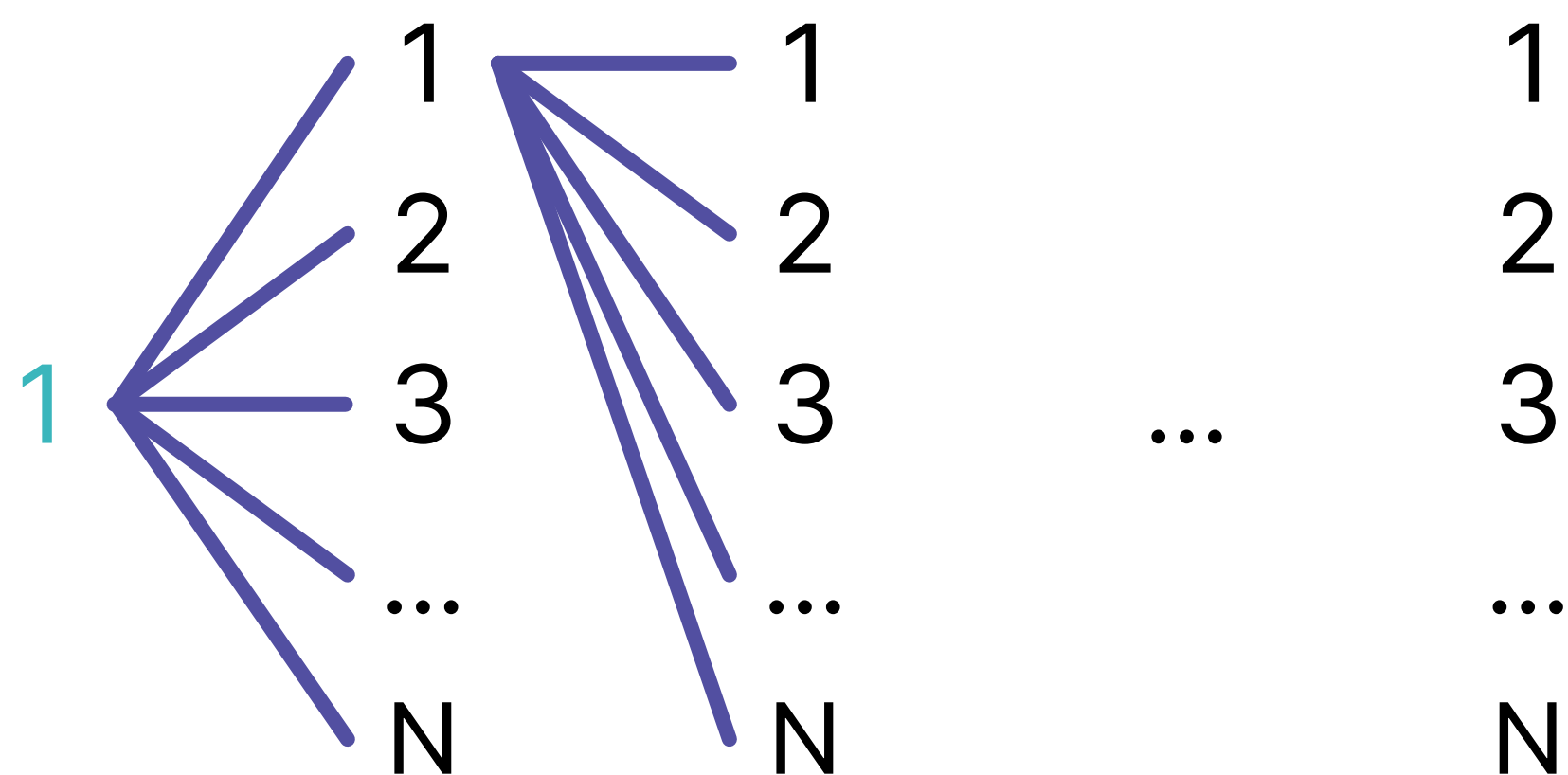
백트래킹 (Backtracking)



그런데, 동작 과정을 잘 살펴보면?



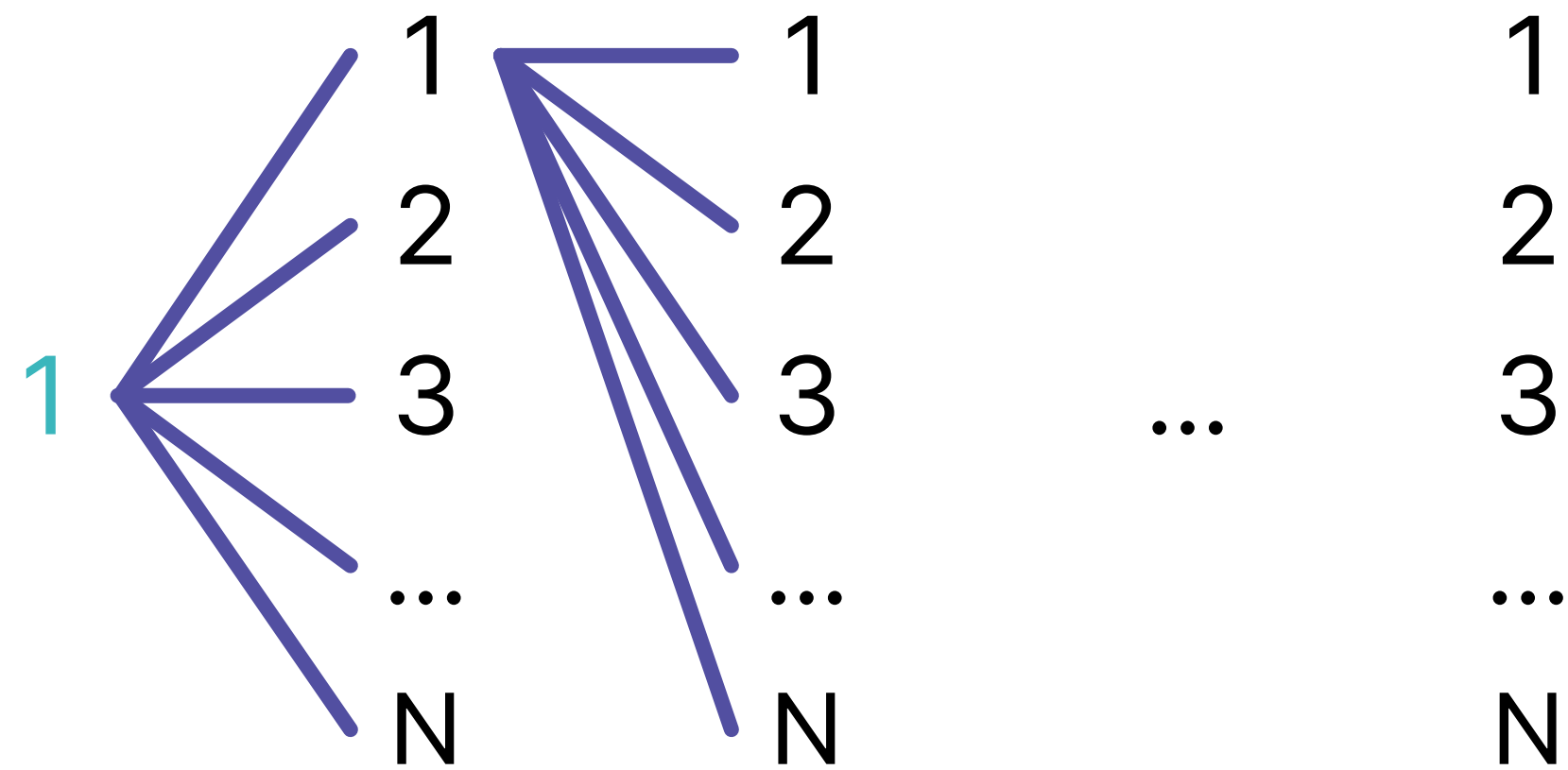
백트래킹 (Backtracking)



순열의 정의가 무엇인지 기억 하시나요?



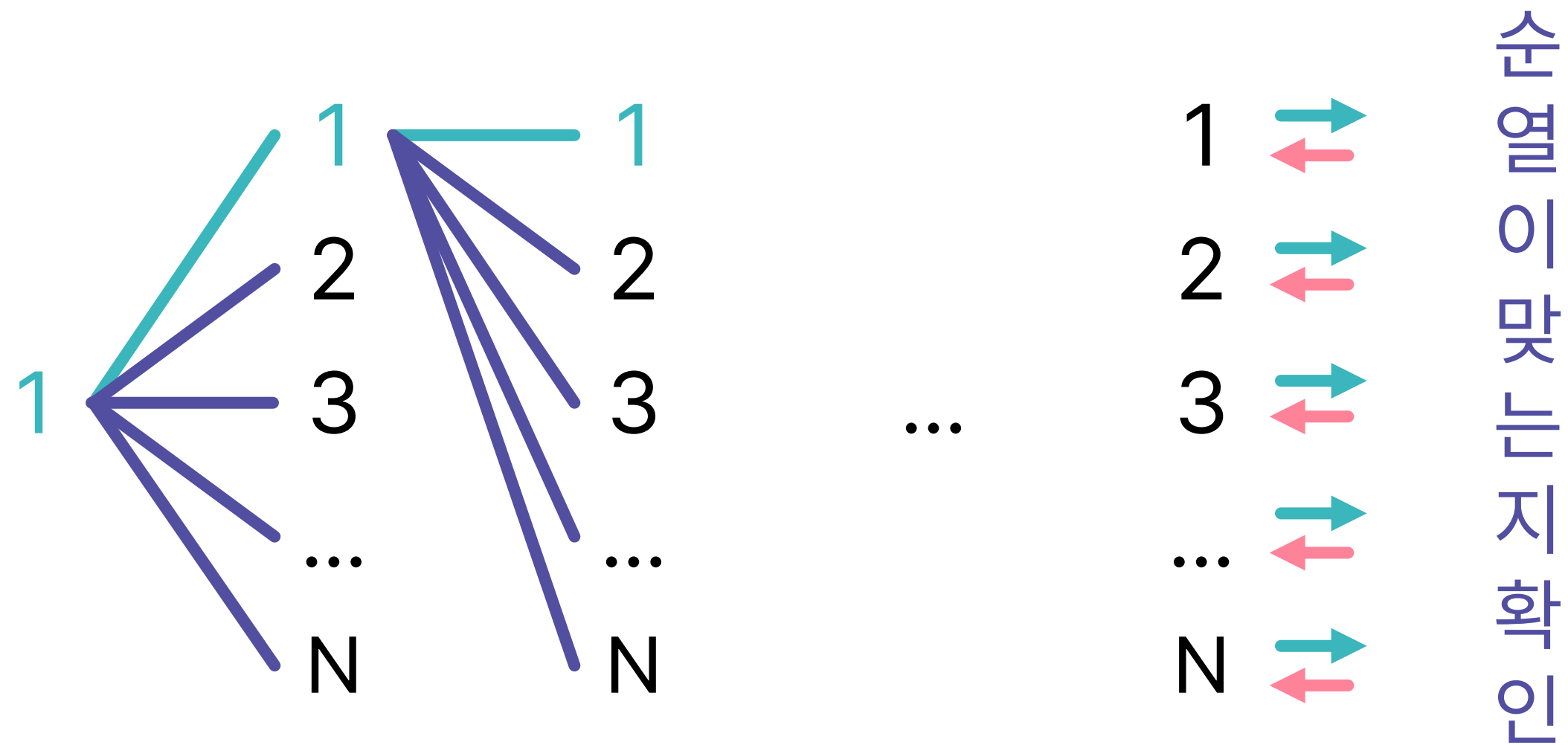
백트래킹 (Backtracking)



특정 원소에서 일부 원소를 **중복없이** 골라
순서에 상관있게 나열하는 것



백트래킹 (Backtracking)

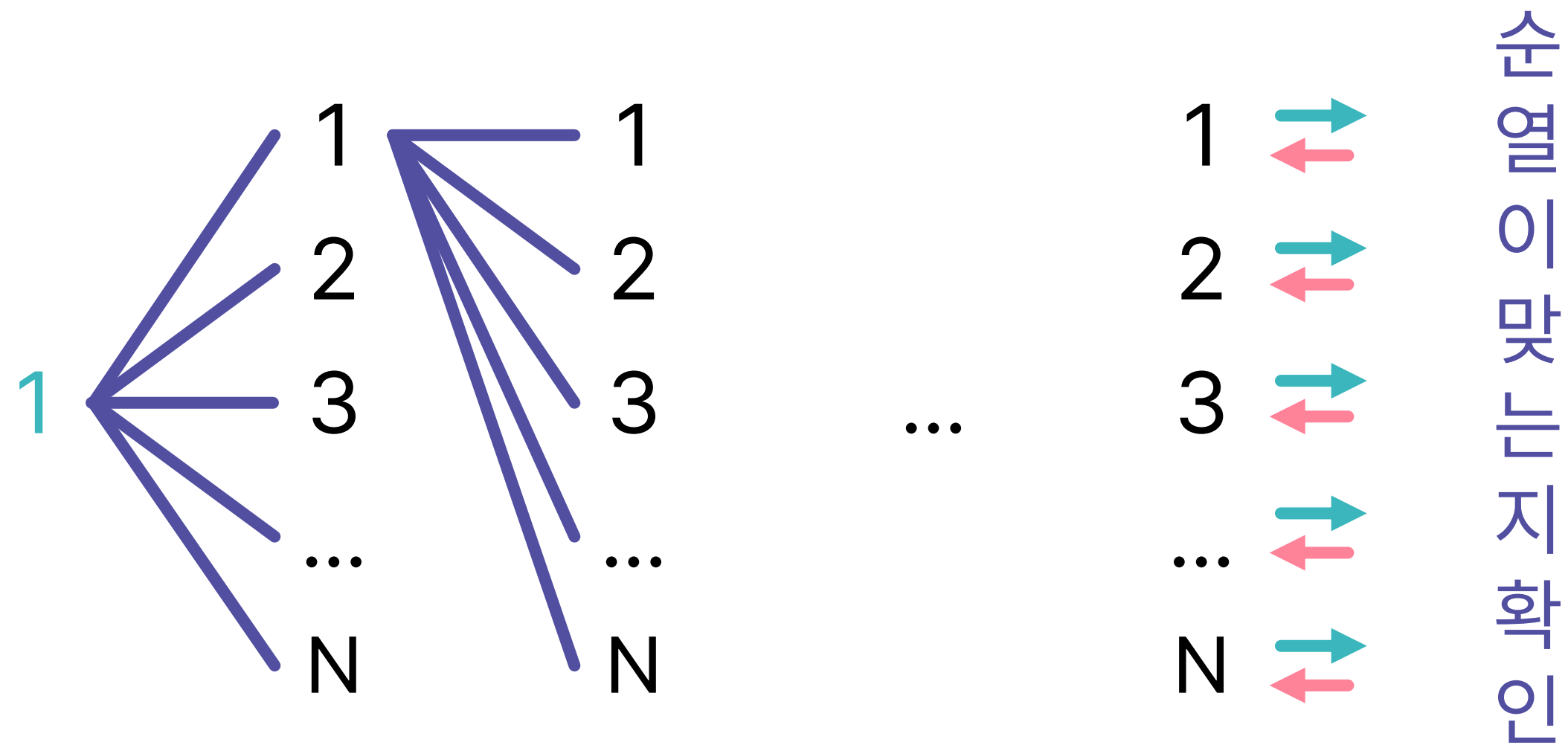


그래서 코드를 작성할 때,

순열인지 확인하기 위해 같은 숫자가 포함되었는지 확인해야 함



백트래킹 (Backtracking)

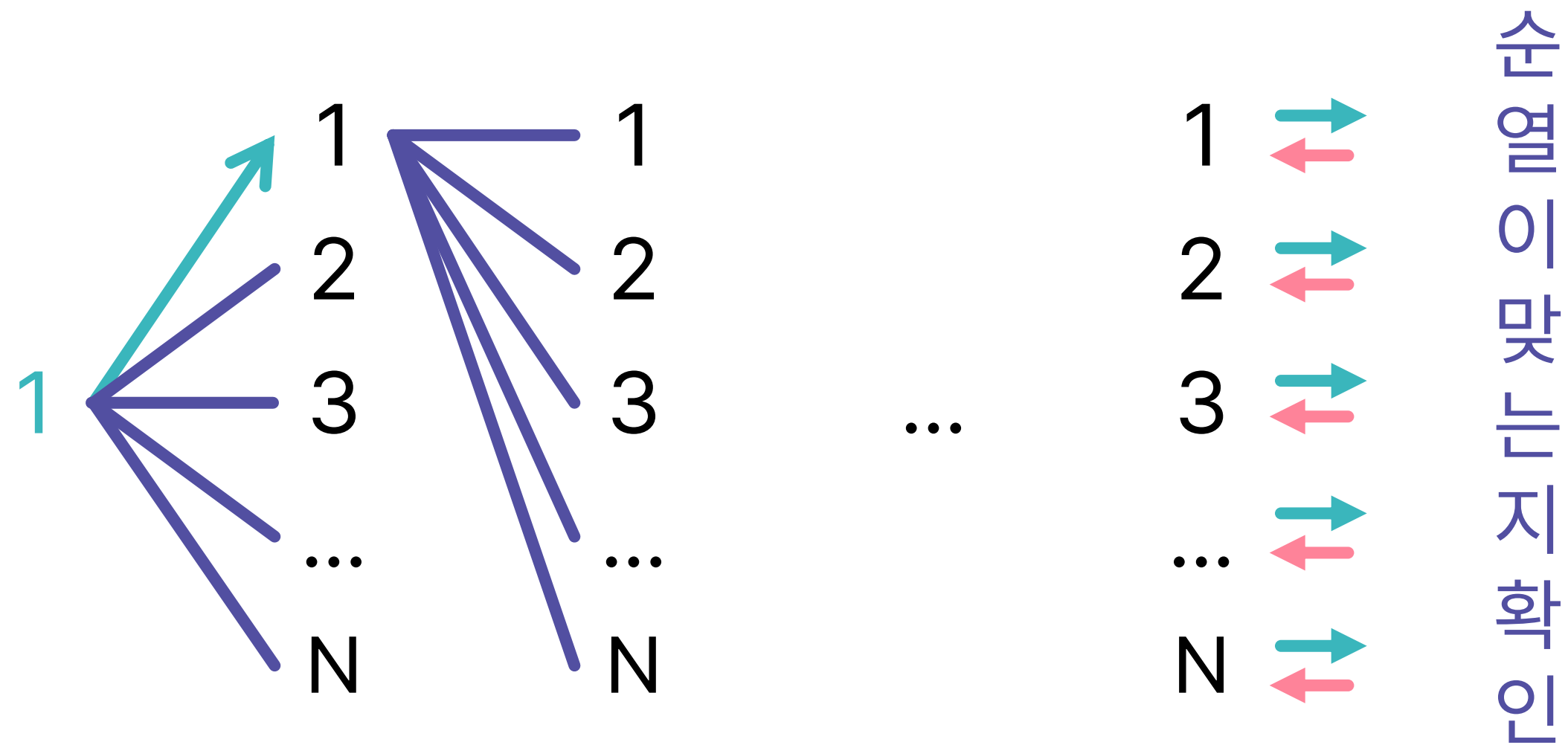


그런데, 맨 끝 까지 갈 필요 없이

수를 추가할 때 마다 중복된 숫자가 있는지 확인하면 안 될까?



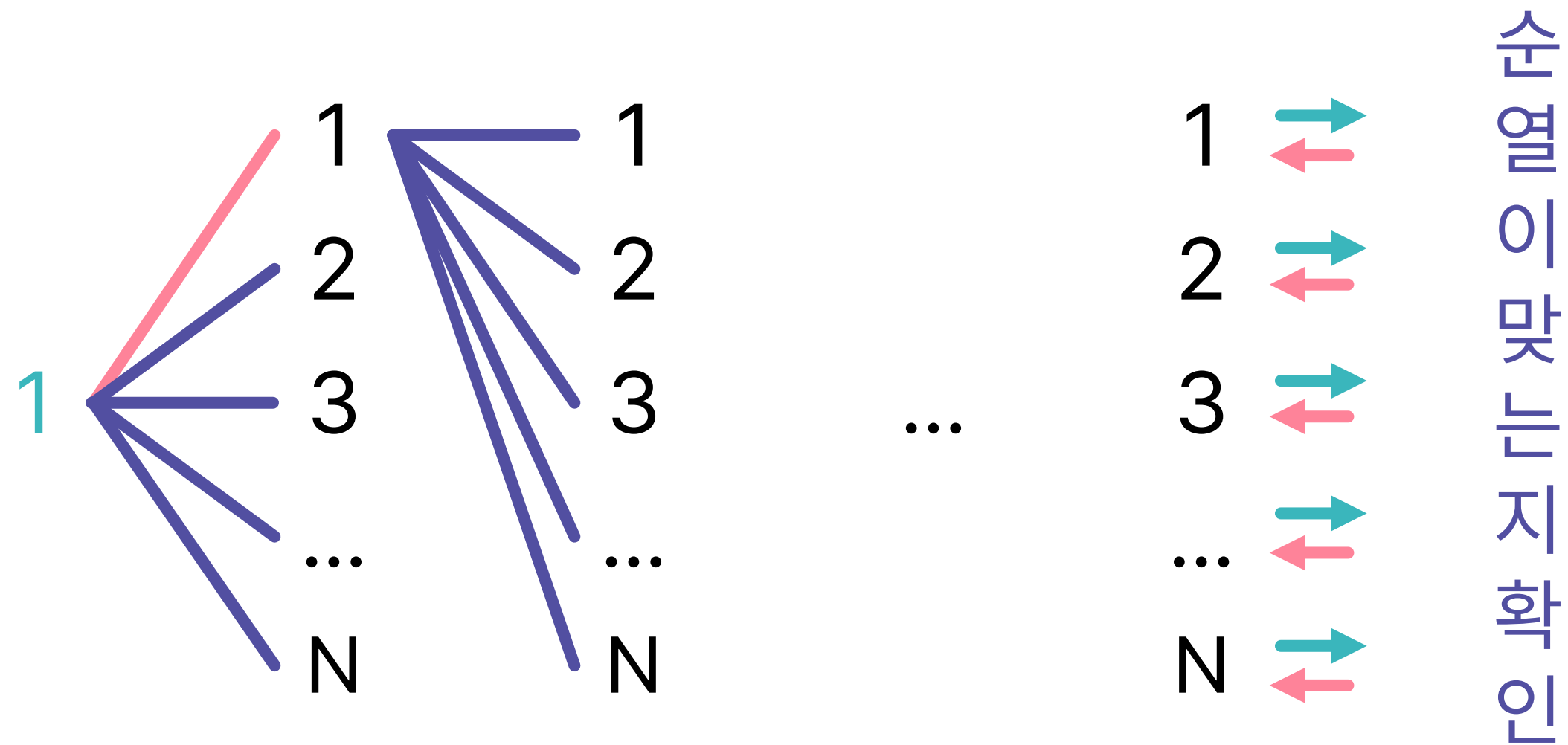
백트래킹 (Backtracking)



예를 들어, 처음으로 선택된 숫자가 1인데,
1을 또 넣으려고 시도한다고 가정해 보자



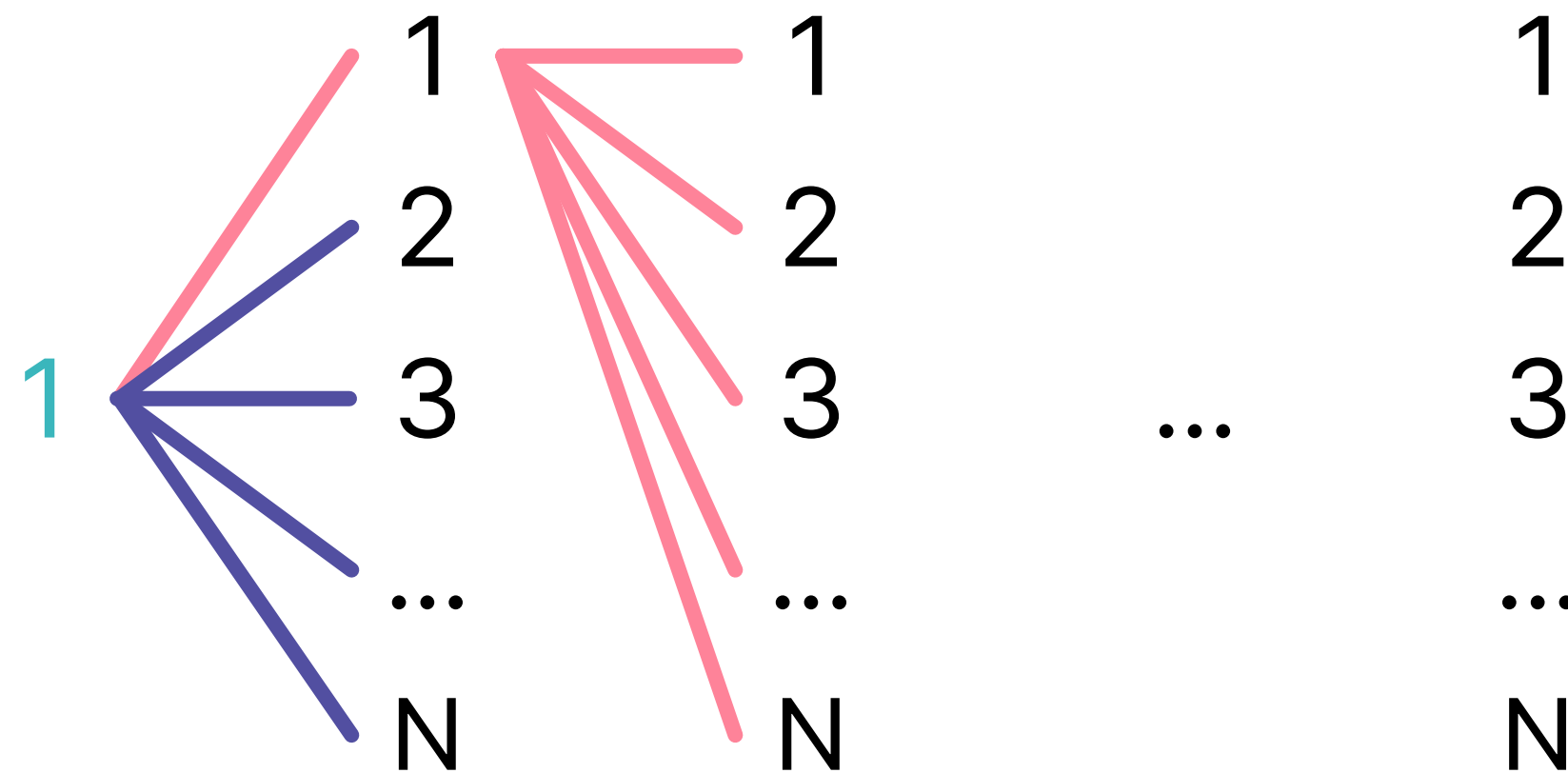
백트래킹 (Backtracking)



그렇다면, 이미 배열에 1이 포함되어 있다는 것을
1을 추가로 넣기 전 미리 확인할 수 있음



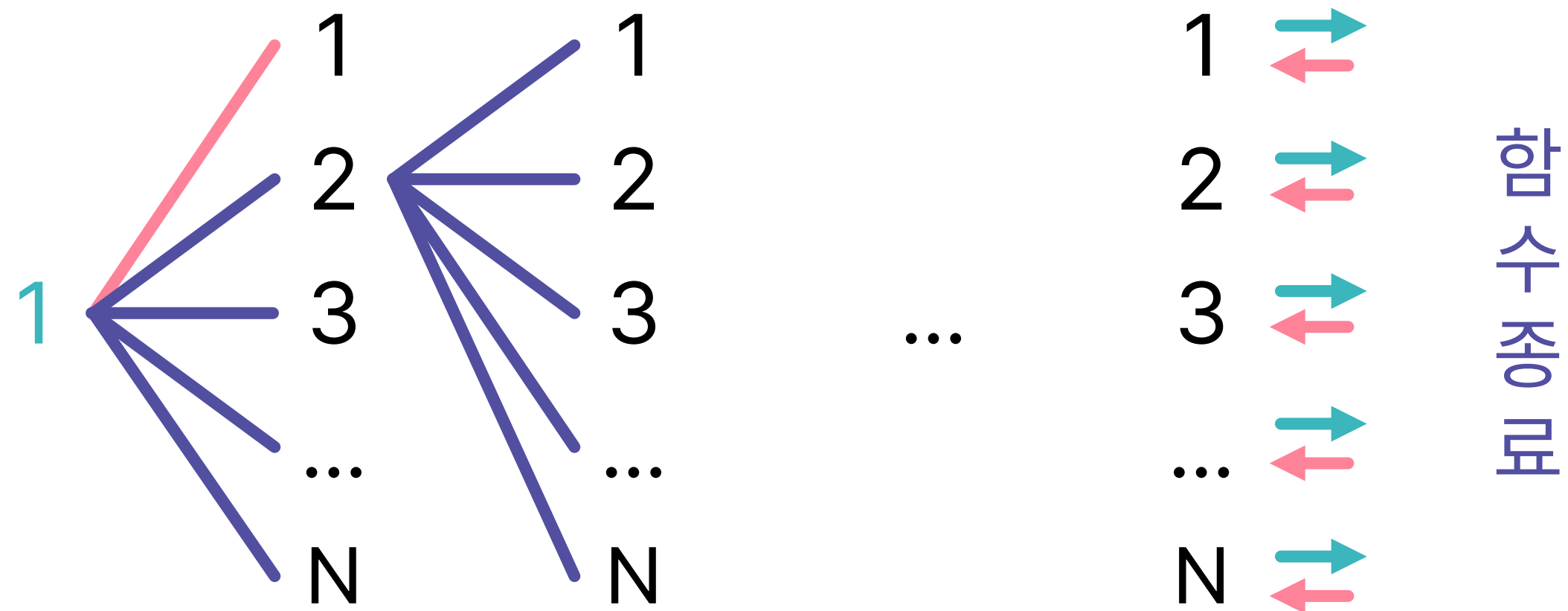
백트래킹 (Backtracking)



1을 고른 상태에서 1을 또 고를 수는 없으니,
뒤에 붙어있는 다른 모든 선택지는 전부 순열이 될 수 없음



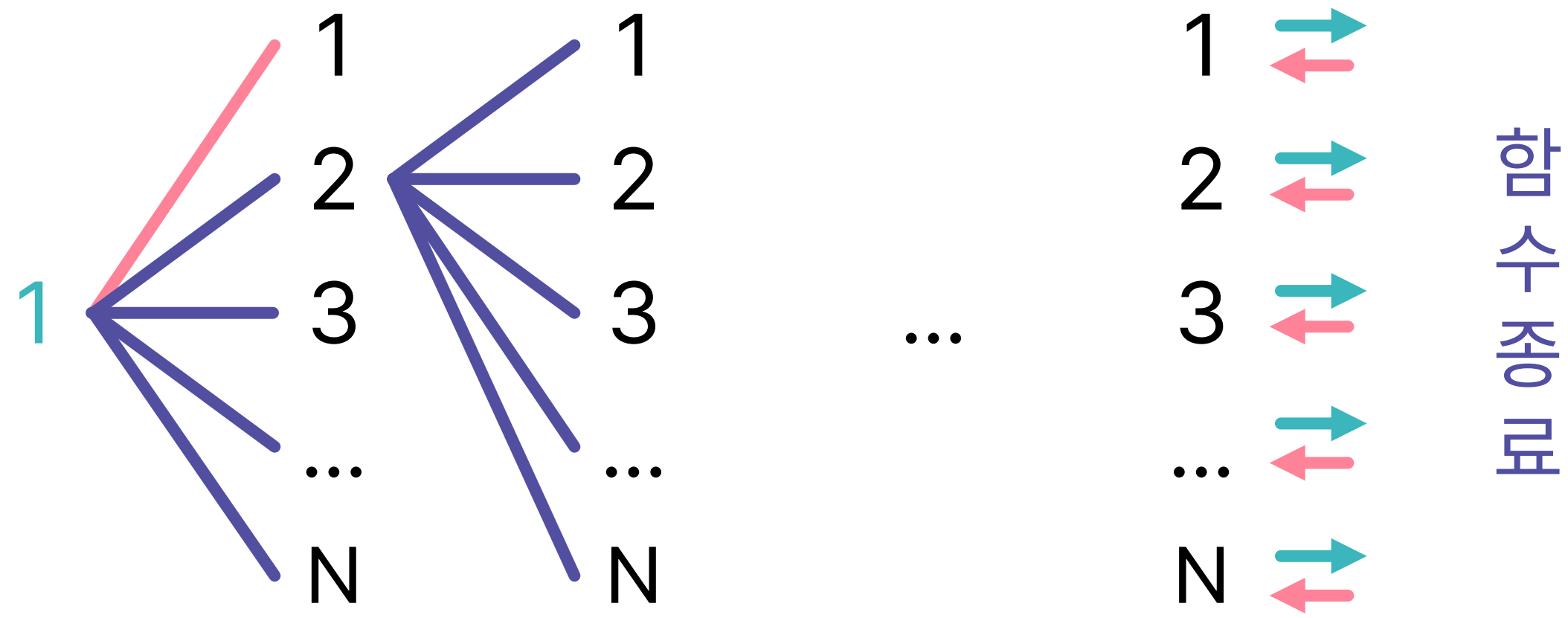
백트래킹 (Backtracking)



그렇다면 1 – 1 처럼 조건에 맞지 않는 선택지가 나오면,
이 뒤에 오는 다른 선택지를 모두 무시할 수 있지 않을까?



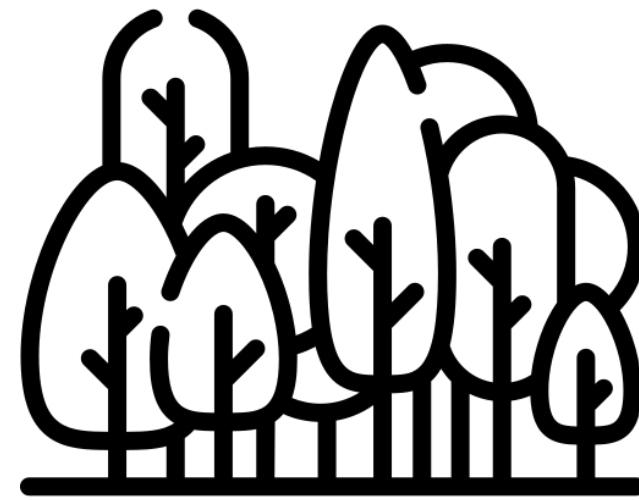
백트래킹 (Backtracking)



이런 식으로 불가능한 경우에 탐색을 중단하고 넘어갈 경우,
경우의 수를 엄청나게 줄일 수 있음



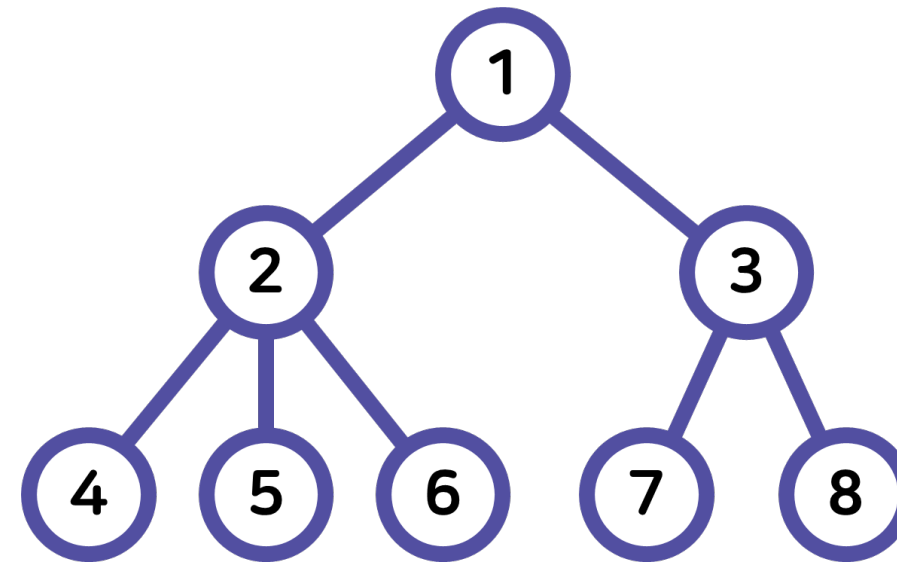
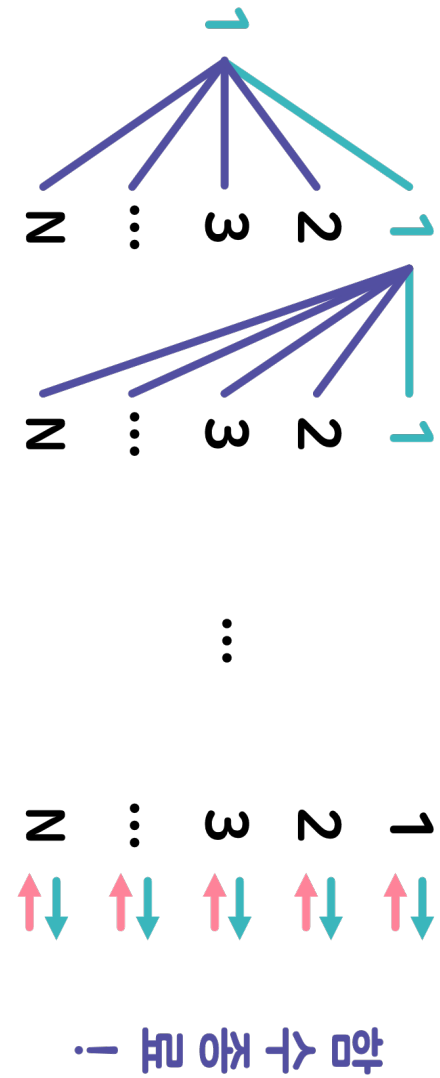
백트래킹 (Backtracking)



우리가 나무를 관리할 때,
나무의 잔가지를 쳐내서 관리하는 것을 가지치기라고 부름



백트래킹 (Backtracking)



앞에서 확인한 재귀함수로 문제를 풀 때,
트리 자료구조의 모양이 나오게 됨



가지치기라고 부름



백트래킹 (Backtracking)

그리고, 이런 식으로 가지치기를 활용해서
완전탐색 알고리즘의 효율성을 높이는 방법을
백트래킹 알고리즘이라고 부름



백트래킹 (Backtracking)

백트래킹 알고리즘으로 문제를 풀 때는,
필요 없는 케이스를 탐색하지 않도록 잘 쳐내야 하며,
필요한 부분을 쳐내지 않도록 조건 설정을 잘 해야 함



백트래킹 (Backtracking)

- **답이 될 수 없는 경우는 탐색 대상에서 제외**하여 효율적으로 답을 구하는 알고리즘
 - 가지치기(pruning)를 통해 **연산량을 유의미하게 줄여줌**
- 가지치기를 사용하기 위해서는 **현재 상태에서 도달할 수 있는 상태가 모두 답이 될 수 없음**을 보여야 함
- 정확한 시간 복잡도를 구하기 어려움





백트래킹 (Backtracking)

예시 - 스도쿠(Sudoku)

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

가로, 세로, 3×3 칸 모두에,
1에서 9까지의 숫자를 중복되지 않게 한 번씩 사용해야 함

스도쿠를 하다 보면, 명확한 근거 없이 일단
대입해 봐야 하는 **분기점**이 존재

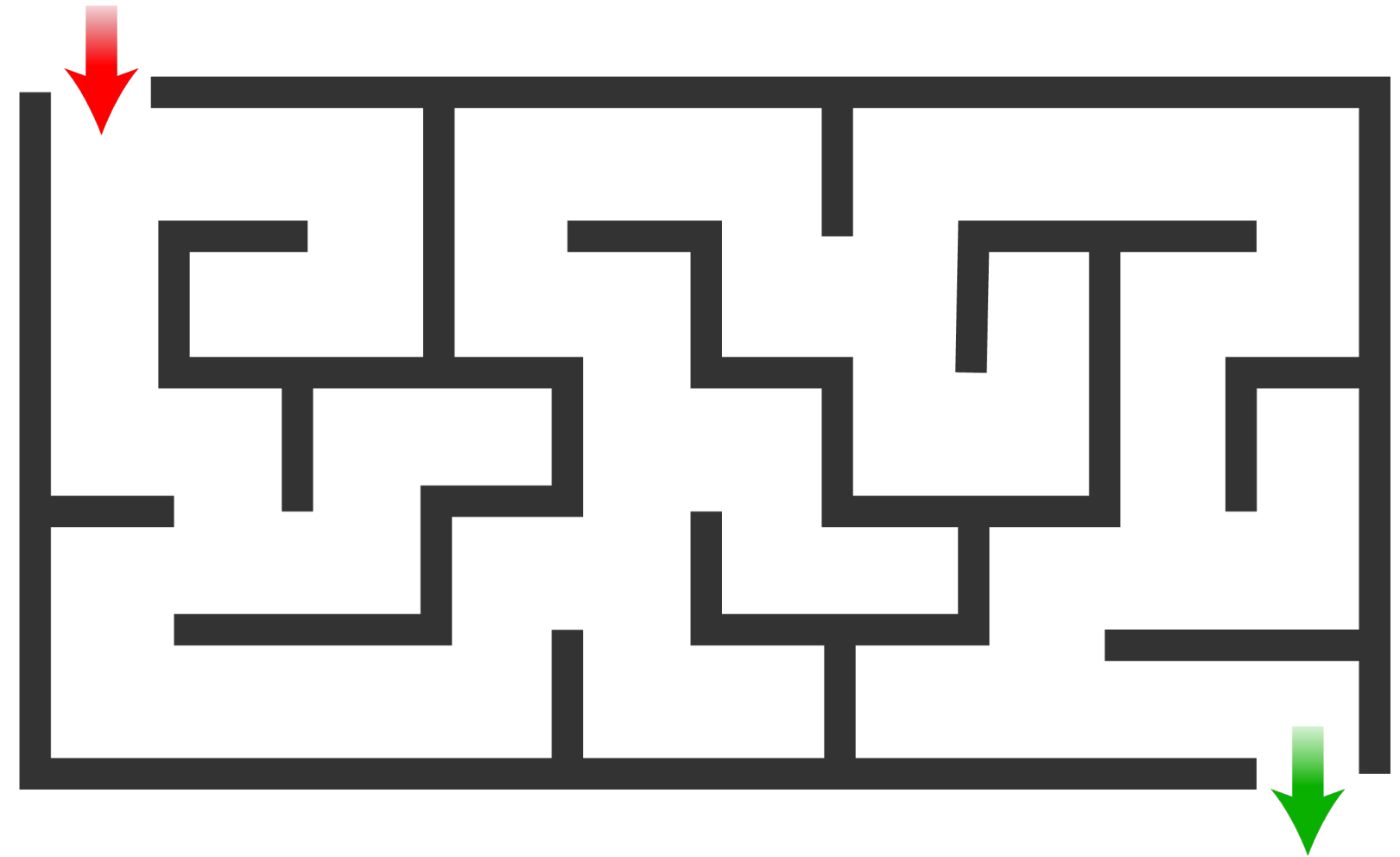
대입해 보고 현재 상태에서 스도쿠를 완성할 수 없다면,
분기점으로 다시 돌아옴 → **백트래킹**



백트래킹 (Backtracking)

분할 정복과의 차이

- 공통점
 - 주로 재귀적인 방식으로 해결
- 차이점
 - 분할 정복은 하위 문제를 해결하고, 결과를 결합하여 문제 해결
 - 백트래킹은 문제 해결을 위해 모든 가능한 선택을 시도한 후, 가능성이 없다고 판단되면 이전 단계로 되돌아가거나 이전 결정을 변경





next_permutation

```
from typing import List

def next_permutation(a: List[int]) -> bool:
    """
    a를 사전순(왼쪽부터 비교)으로 '다음 순열'로 바꾼다.
    예를들어, [1, 2, 3] -> [1, 3, 2] -> [2, 1, 3] -> ... -> [3, 2, 1]
    다음 순열이 존재하면 True, 마지막 순열이면 False (그리고 a는 첫 순열로 리셋됨).
    """
    n = len(a)
    if n < 2:
        return False

    # 1) 뒤에서부터 '증가가 깨지는 지점' 찾기:  $a[i] < a[i+1]$ 인 가장 오른쪽  $i$ 
    i = n - 2
    while i >= 0 and a[i] >= a[i + 1]:
        i -= 1

    # 이미 내림차순이면 마지막 순열 -> 첫 순열(오름차순)로 리셋
    if i < 0:
        a.reverse()
        return False
```



next_permutation

```
# 2) 뒤에서부터 a[i]보다 큰 값 중 가장 오른쪽 j 찾기 (a[i+1:] 는 내림차순)
j = n - 1
while a[j] <= a[i]:
    j -= 1

# 3) swap
a[i], a[j] = a[j], a[i]

# 4) a[i+1:] 구간을 가장 작게 만들기 (오름차순으로 만들어 최소 변화 만들기)
a[i + 1:] = reversed(a[i + 1:])
return True
```